Graphic Era Hill University
DEHRADUN • BHIMTAL • HALDWANI

Secure Suite by Gauresh & Ayush

Second Phase Project Report

Project Title

Secure Suite

Student/Team Information

Team Name:	Avengers
Team member 1 (Team Lead) Name: Gauresh Student ID: 0 Email:	
Team member 2 Name: Anshika Rana Student ID: 25210820177 Email: ANSHIKARANA.25210820177@gehu.ac.in	

PROJECT PROGRESS DESCRIPTION

Project Abstract

Secure Suite is a comprehensive, GUI-based cybersecurity tool designed to simplify network monitoring, vulnerability detection, and threat intelligence gathering. It integrates multiple open-source security tools (e.g., Nmap) and cloud-based APIs (e.g., VirusTotal) into a single dashboard, enabling security analysts and system administrators to detect suspicious activity, identify malware indicators, and receive early warnings of "fishy" behavior on their network.
Managing network security often requires switching between multiple command-line tools, interpreting disparate outputs, and manually correlating threat data. This fragmented approach is time-consuming, error-prone, and unsuitable for rapid incident response. There is a need for a unified platform that:

- Aggregates scanning and monitoring capabilities.
- Automates threat intelligence lookups.
- Presents results in an intuitive graphical interface.
- Flags anomalies without deep command-line expertise.

Updated Project Approach and Architecture

- Language: Java
- Compiler: Javac
- Standard Libraries: Java awt , swing,bufferreader
- Python Scripts

User Interface (Java Swing , AWT) ->
->Core Logic (Java and Python) ->
->System Calls(Powershell execution of programs)

Tasks Completed

Task Completed	Team Member
User Interface , Resource Gathering , Python Scripts for tools , Testing.	Ayush Bisht
Project Design , Backend (API ,System Calls, Multi-Threading), Documentation.	Gauresh Bhatt

Challenges/Roadblocks

1.Multi-threading is not fully implemented
Multi-threading was attempted across all scanning and monitoring functions, but it became too complex to manage reliably. As a result, some operations (like long Nmap scans or VirusTotal API lookups) can temporarily freeze the GUI until they complete.
2. Complex OOPS design in a single file
The project uses heavy object-oriented programming, but all classes and functions are written inside one large Python file. There is no clear separation of modules or folders. This makes the code hard to navigate — finding a specific class or function takes too much time, even for someone familiar with the code.

Tasks Pending

Task Pending	Team Member (to complete the task)
UI UI Sync Backend Improve Scripts , add more tools	Ayush Gauresh Gauresh-Ayush

Project Outcome/Deliverables

- The project successfully provides a graphical interface that allows users to run security scans, view live network data, and receive alerts without using command-line tools. The dashboard displays active hosts, open ports, and service details in a color-coded table
  Nmap – Fully integrated. Users can run quick scans, full port scans, and OS detection directly from the GUI. Results are parsed and displayed automatically.
- VirusTotal API – Fully integrated. Users can submit file hashes, IP addresses, or domain names and receive threat reports showing how many antivirus engines flagged the item as malicious.

The project includes three custom scripts that run alongside the GUI:

- Log monitor – Detects repeated failed login attempts (brute-force indicator).
- Packet sniffer – Flags unusual protocols or unexpected external connections.
- Baseline comparator – Compares current open ports against a saved baseline and alerts when new ports appear.

Progress Overview

What Works (Current Project Status)

1. GUI – Mostly Functional

The graphical user interface launches successfully and all main windows are accessible. The dashboard is present and displays core information, though it currently remains static (not yet fully populated with live data). Navigation between different sections of the application works as expected.

Status: GUI works, but dashboard remains (needs live data integration).

---

2. OSINT Window – Not Working (Needs Fix)

The OSINT (Open Source Intelligence) window is currently non-functional. This window was intended to aggregate threat intelligence from public sources, but it does not load or respond properly. This is a known issue that requires debugging and fixing.

Status: Does not work – needs to be fixed.

---

3. Multi-Threading – Works

Unlike the earlier project description, multi-threading has been successfully implemented. Long-running operations such as Nmap scans and API calls now run in background threads, which means the GUI remains responsive and does not freeze during these tasks. This is a major improvement over the previous version.

Status: Fully functional.

---

4. Python Tool Scripts – Work but Need Improvement

The custom Python scripts (log monitor, packet sniffer, baseline comparator, etc.) run successfully and perform their intended detection functions. However, they require improvement in areas such as:

- Error handling for edge cases
- Performance optimization
- Better integration with the GUI
- More reliable parsing of system logs across different operating systems

Status: Works, but needs improvement.

---

5. Missing Features – Need to Add

The project currently lacks sufficient tools and corresponding GUI windows. Planned additions include:

- More security tool scripts (e.g., WHOIS lookup, DNS enumeration, hash verifier, URL expander)
- Additional GUI windows to host and control these new tools
- Better organization of tools within the dashboard

Status: Need to add more tool scripts and windows.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Network Scan	Passed	Needs to input its own network first
Scan for alert	Failed	Needs to work on different types of scans like decoy and T3
Virus Total APT	Passed	URL checker and Spyware checker works fine

Deliverables Progress

Multi-threading is finally done and working properly. This means when you run an Nmap scan or call the VirusTotal API, the GUI no longer freezes up – you can still click around while the work happens in the background. The basic GUI framework is also complete. You can launch the app, click through the tabs, use the buttons, and the layout looks like it should. The core Python scripts – the log monitor, packet sniffer, and baseline comparator – are all working in their basic form. They run and give you results, no major crashes.

Now, here is what still needs attention.

The dashboard works, but honestly, it just sits there. It has the right structure and looks nice, but it does not show live data yet. You cannot see scans updating in real time. That needs to be added.

The Python scripts work, but they are a bit rough around the edges. Error handling could be better. They could run faster. They do not always play nicely across different operating systems. And they could be integrated more smoothly into the GUI instead of feeling like separate tools bolted on.

Nmap integration works for basic scans, and yes, it runs in the background thanks to multi-threading. But I still need to add more scan types, improve how the output gets parsed, and show some kind of progress indicator so you know how long a scan will take.

VirusTotal integration is functional – you send a hash or IP, and you get threat data back. But there is no caching right now, so the same lookup hits the API over and over. That wastes your quota. Error handling for invalid keys or hitting rate limits is also weak. And the results display could look much better.

The biggest problem is the OSINT window. It simply does not work. That needs to be fixed completely.

And finally, we need more tools. Right now, the project feels a bit empty. More Python scripts and more GUI windows to house them are missing. Without those, the tool is not really feature-complete.